

SUPERVISED MACHINE LEARNING
Multiple Linear Regression

Jean René RANDRIANJAKAMANANA

2024-11-25

Table de matières

II) MULTIPLE LINEAR REGRESSION	2
1) Model ameliorated through Gradient Descent	2
2) Model with sklearn	6

II) MULTIPLE LINEAR REGRESSION

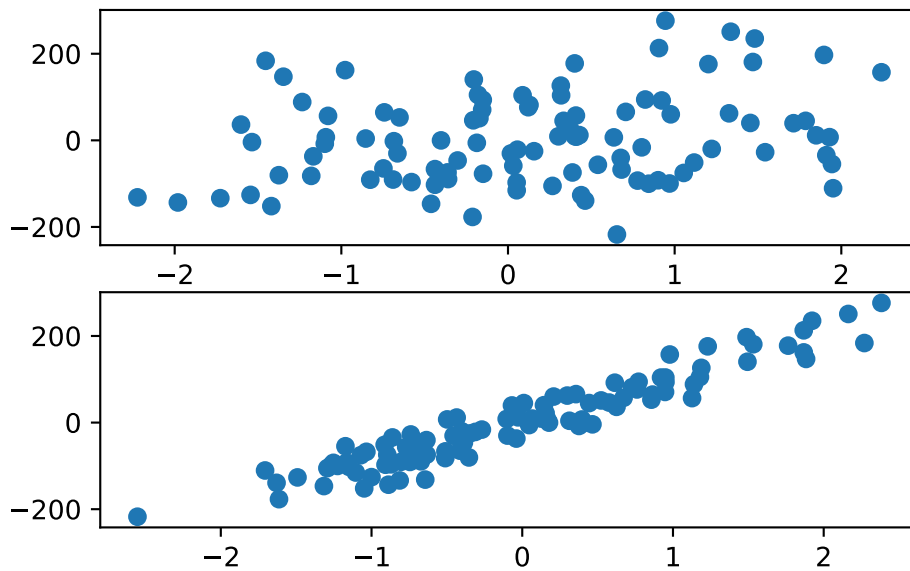
$$y = a_0 + a_1x_1 + \dots + a_nx_n$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from sklearn.datasets import make_regression
```

1) Model ameliorated through Gradient Descent

a) Importation of datasets and plot

```
1 # n_features=2 means that there are two explicatives variables x1 and x2.
2 np.random.seed(0)
3 x, y = make_regression(n_samples = 100, n_features = 2, noise = 10)
4
5 # Representation with the first feature
6 plt.close()
7 plt.subplot(2,1,1)
8 plt.scatter(x[:,0],y)
9
10 # OWith the second feature
11 plt.subplot(2,1,2)
12 plt.scatter(x[:,1],y)
13 plt.show()
```



b) Dimension and resize of data

```

1 x.shape
2 y.shape
3
4 y = y.reshape(y.shape[0],1)
5 y.shape

```

(100, 1)

c) Random estimation of the model's parameters

```

1 # Y = X*beta + epsilon
2 #Y_ajust = X*beta_ajustr
3 np.random.seed(0)
4 X = np.hstack((x, np.ones((x.shape[0], 1))))
5
6 # Creation of matrix X with constant X[:,10]
7 beta = np.random.randn(3,1)
8 beta

```

```

array([[1.76405235],
       [0.40015721],
       [0.97873798]])

```

d) Model function creation and Regression plot

```

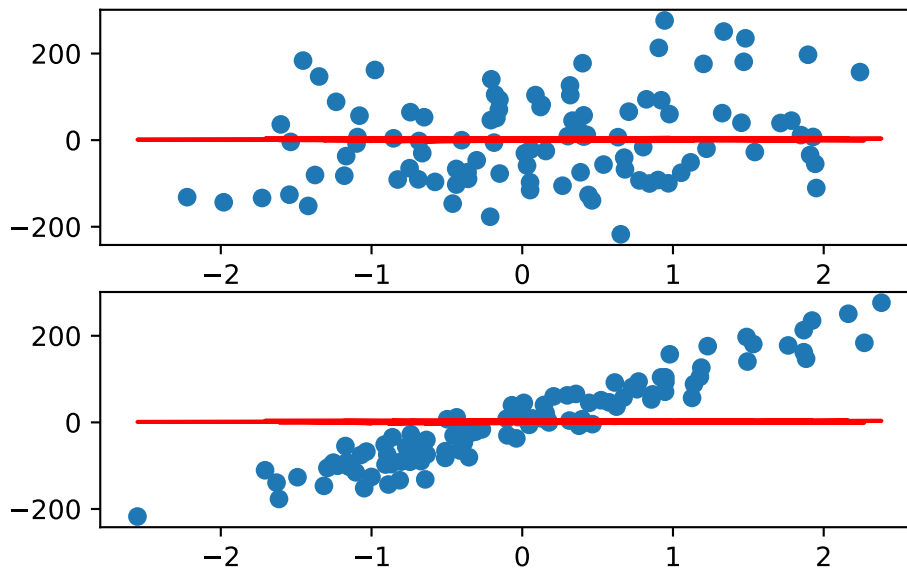
1 def modele(X,beta):
2     return X.dot(beta)
3
4 plt.close()

```

```

5 plt.subplot(2,1,1)
6 plt.scatter(x[:,0],y)
7 plt.plot(x[:,1], modele(X, beta), c='r')
8 plt.subplot(2,1,2)
9 plt.scatter(x[:,1],y)
10 plt.plot(x[:,1], modele(X, beta), c='r')
11 plt.show()

```



e) Loss function creation

```

1 def cost_function(X, y, beta):
2     m = len(y)
3     return (1/2*m)*np.sum((modele(X,beta) - y)**2)
4
5 cost_function(X,y,beta)

```

52587594.45573963

f) Minimisation of loss function: Gradient Descent

```

1 #Fonction gradient
2 def grad_cost_func(X, y, beta):
3     m = len(y)
4     return (1/m)*X.T.dot(modele(X,beta) - y)
5
6 #Algorithme de gradient descent
7 def gradient_descent(X,y,beta, learning_rate, n_iterations):
8     cost_history = np.zeros(n_iterations)
9     for i in range(0, n_iterations):
10         beta = beta - learning_rate*grad_cost_func(X,y,beta)

```

```

11 cost_history[i] = cost_function(X,y,beta)
12 return beta, cost_history

```

g) Model training

```

1 beta_final, cost_history = gradient_descent(X,y,beta, learning_rate = 0.1, n_iterations = 1000)
2 beta_final

```

```

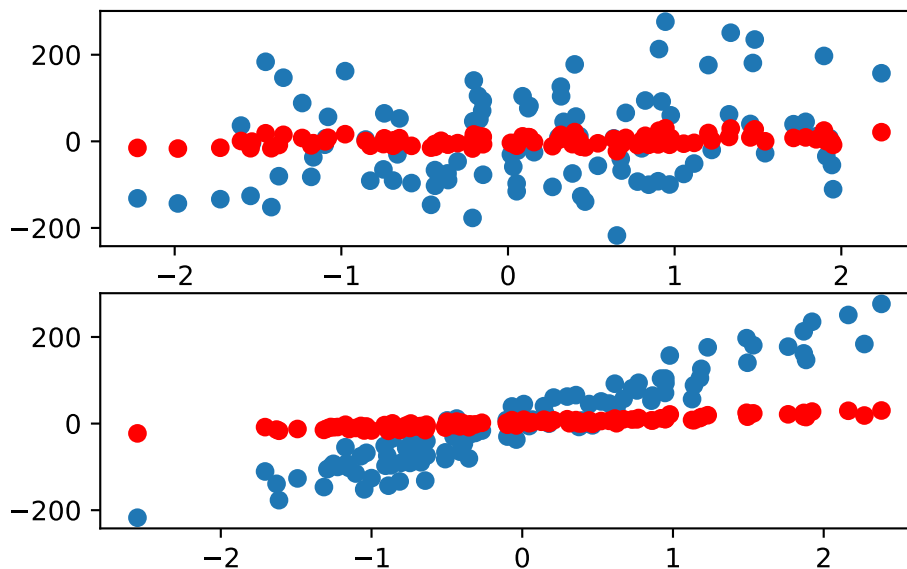
array([[ 4.33162401],
       [10.45290048],
       [ 1.2043954 ]])

```

```

1 y_predi = modele(X, beta_final)
2
3 plt.close()
4 plt.subplot(2,1,1)
5 plt.scatter(x[:,0], y)
6 plt.scatter(x[:,0], y_predi, c='r')
7 plt.subplot(2,1,2)
8 plt.scatter(x[:,1], y)
9 plt.scatter(x[:,1], y_predi, c='r')
10 plt.show()

```

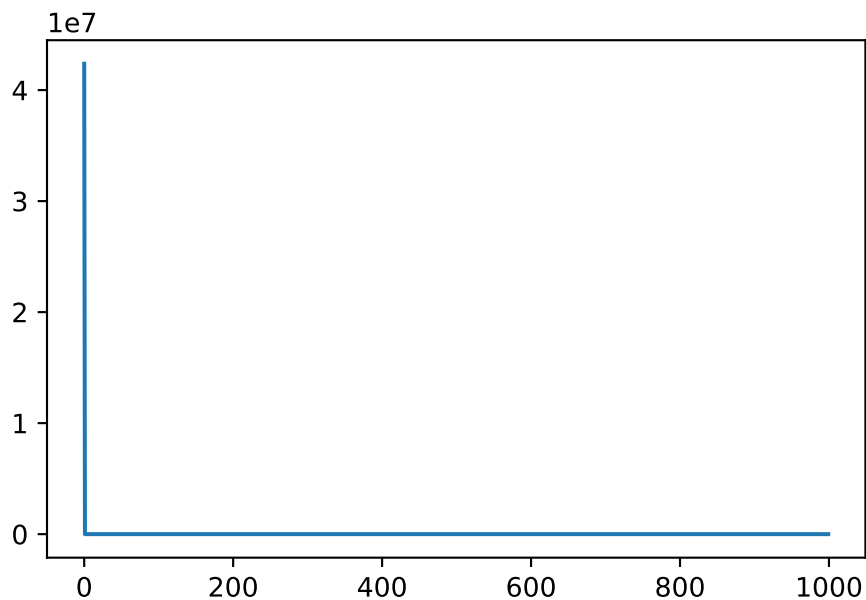


h) Learning curve (Decreasing plot of the loss function)

```

1 plt.close()
2 plt.plot(range(1000), cost_history)
3 plt.show()

```



i) Coefficient of determination (R^2)

```

1 def coeff_determination(y, y_predi):
2     u = ((y-y_predi)**2).sum()
3     v = ((y-np.mean(y))**2).sum()
4     return (1-u/v)
5
6 R2 = coeff_determination(y, y_predi)
7 print(f"R2 = {R2} or {round(R2,5)*100} %")

```

$R^2 = 0.20689156542250542$ or 20.689 %

2) Model with sklearn

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from sklearn.datasets import make_regression
5
6 from sklearn.linear_model import LinearRegression
7 from sklearn.metrics import mean_squared_error, r2_score
8
9 np.random.seed(0)
10 x, y = make_regression(n_samples = 100, n_features = 2, noise = 10)
11
12 x.shape
13 y = y.reshape(y.shape[0], 1)
14 regression = LinearRegression()
15 regression.fit(x,y)

```

```

9 y_pred = regression.predict(x)
10 regression.score(x,y)
11
12 regression.coef_
13 regression.intercept_
14
15 plt.close()
16 plt.subplot(2,1,1)
17 plt.scatter(x[:,0], y)
18 plt.scatter(x[:,0], y_pred)
19
20 plt.subplot(2,1,2)
21 plt.scatter(x[:,1], y)
22 plt.scatter(x[:,1], y_pred)
23 plt.show()

```

